

Control laws in the Rattlesnake MIMO random environment

Algorithms, equations and flow charts · 6-12-8 system · 15 September 2026 · `control_laws/`

Seven subjects: the six control laws compared on hardware, plus the achievable-response predictor that supplies the denominator every comparison is measured against. Each section gives the algorithm, the governing equations, a flow chart, the parameter string, and what was actually measured. Sections 5 and 6 cover the two original upstream open-loop laws, added 15 September 2026 after runs 09–12; section 7 covers the drive-coherence cap, whose meaning turns out to depend on which family it is applied to.

0. Common interface and notation

Rattlesnake loads a control law by explicit path with `importlib.util.spec_from_file_location` — there is no package context, so a law cannot use relative imports. A law is either a plain function called once per control cycle, a generator, or a class constructed at `INITIALIZE_PARAMETERS` (before system ID) whose `control()` is then called each cycle. The profile names the script, the function, and the parameter string; the function type is inferred from the module.

Every law returns the same object: the commanded drive CPSD.

`N` drives (6 here) `M` control channels (8) `F` frequency lines (901 in band)

$H(f)$ $M \times N$ frequency response function

$X(f)$ $N \times N$ drive CPSD – what the law returns; must be ≥ 0

$G_{xx}(f)$ $M \times M$ specification CPSD (diagonal here)

$Y(f) = H X H^H$ predicted response CPSD

$y_m = \text{diag}(Y)_m$ the auto-spectra that are actually specified and scored

The constraint that shapes every law.

X must be a valid cross-spectral matrix, i.e. positive semidefinite. The set of achievable response auto-spectra is therefore $D = \{ \text{diag}(H X H^H) : X \succeq 0 \}$, the image of the PSD cone under a linear map. When $M > N$ — eight control channels from six shakers — D is a *proper* subset of the positive orthant, so a perfectly reasonable flat specification can be partly unreachable. No law gets outside D . How each law respects positive semidefiniteness is the main thing that distinguishes them.

1. match_trace_pseudoinverse

function · control_laws.py

The baseline, and the only law in the set with a long validation history. One open-loop pseudoinverse solve at startup, then a single real scalar per frequency line for the rest of the test.

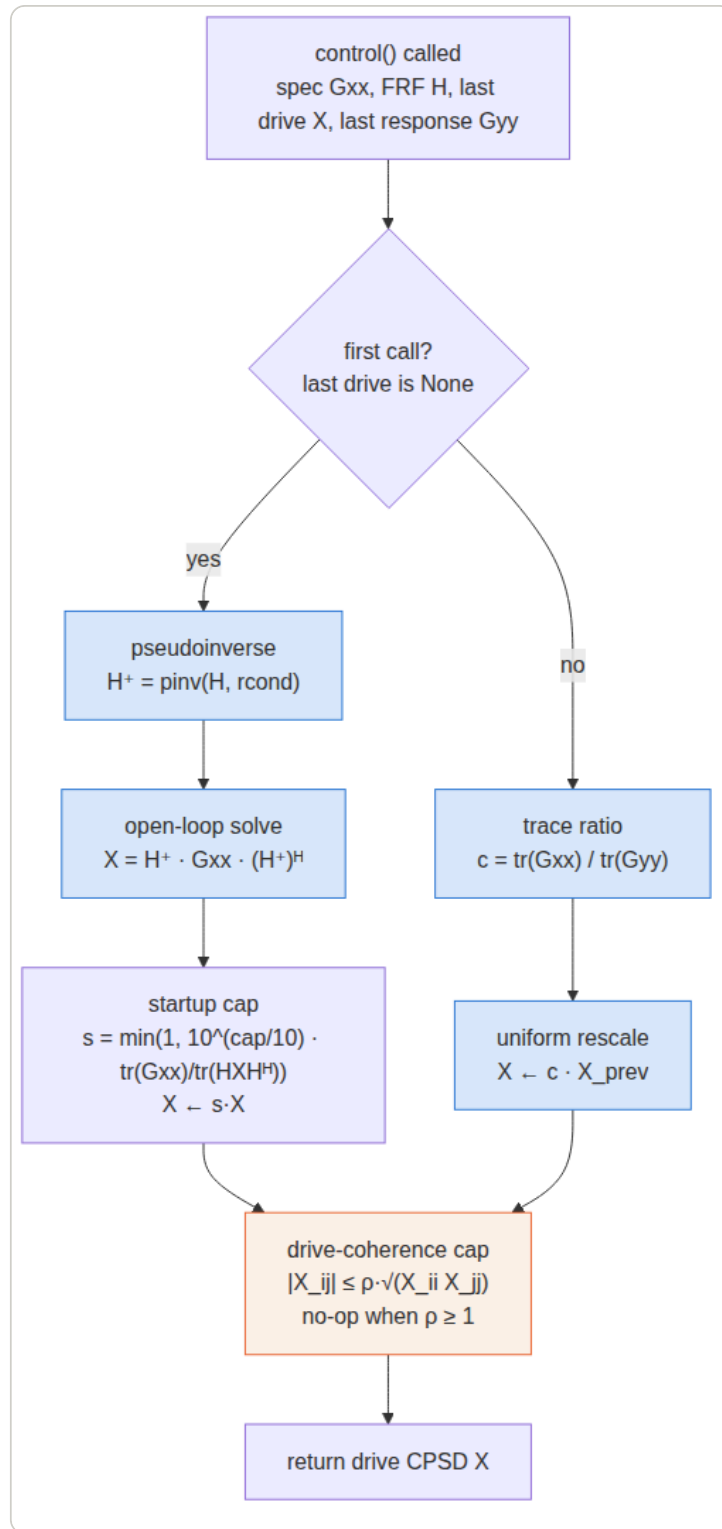
Algorithm

```
first call – open loop
H+ = pinv(H, rcond)
X = H+ Gxx (H+)H
s = min(1, 10cap/10 · tr(Gxx) / tr(H X HH))  startup guard
X ← s · X

every call after that – closed loop on total power only
c(f) = tr(Gxx(f)) / tr(Gyy(f))
X(f) ← c(f) · Xprev(f)
```

The update is, in log-power terms, a **pure discrete integrator with gain 1** on the log-power error: writing $e(k) = \log \text{tr}(G_{xx}) - \log \text{tr}(G_{yy}(k))$, the update is exactly $\log X(k) = \log X(k-1) + e(k-1)$. There is no proportional term, and the error it integrates arrives through whatever CPSD averaging is configured — a pure integrator fed a lagged error, which is the classic slow-to-settle, overshoot-prone configuration. Simulation at averaging coefficient 0.10 gave ~53 cycles and ~103 % peak overshoot to settle within 5 %.

Why it is safe: a uniform positive real scalar can never turn a valid drive CPSD into an invalid one, and after startup the FRF is never inverted again. **Why it is limited:** if channel 3 is 5 dB high and channel 5 is 5 dB low, the trace is correct and the law does nothing at all.



`match_trace_pseudoinverse`. The only per-cycle computation is one scalar per frequency line.

Parameters

POSITION	NAME	DEFAULT	MEANING
1	rcond	1e-15	Pseudoinverse conditioning cutoff. Affects the startup solve only. 1e-3 was used in this study.
2	max_drive_coherence	1.0	Cap on pairwise drive coherence. 1.0 is a true no-op — the function returns the CPSD unchanged.
3	startup_test_level_cap_db	-9.0	Ceiling on the first command's predicted response trace, relative to the spec trace.

2. optimal_diagonal_control

class · optimal_diagonal_control.py

Chooses the drive CPSD's cross-terms to minimise diagonal error directly, rather than fixing them from an independent-drive survey. Per frequency line it solves a genuine semidefinite program.

The per-bin SDP

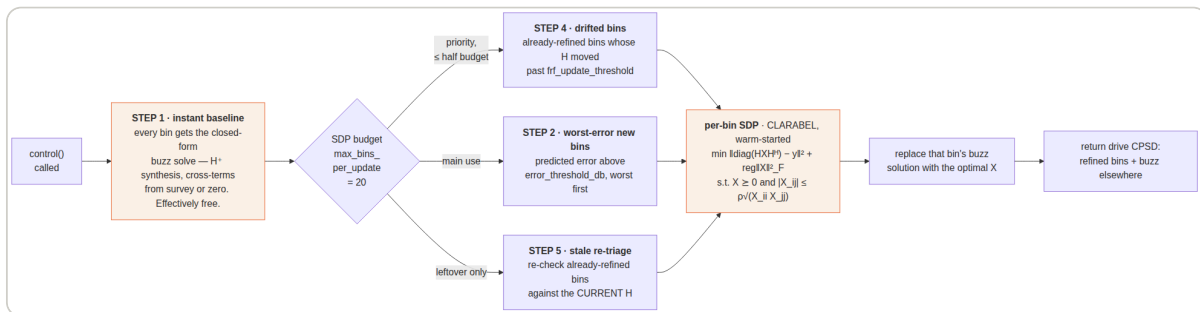
$$\begin{aligned} \text{minimize}_X \quad & \| \text{diag}(H X H^H) - y_{\text{target}} \|^2 + \text{reg} \cdot \|X\|_F^2 \\ \text{subject to} \quad & X \geq 0 \\ & |X_{ij}| \leq \rho \cdot \sqrt{X_{ii} X_{jj}} \quad \text{for all } i \neq j \end{aligned}$$

Both constraints are convex — the coherence one is $| \text{affine} | \leq \text{concave}$ — so this is a true convex program with no manifold optimisation, solved with CLARABEL and warm-started between calls.

The budgeted scheduler

Solving every line up front costs seconds. Instead the law runs a five-step schedule per call, spending a fixed budget of `max_bins_per_update` SDP solves where they do most good, so control can begin on iteration 1 with a full-spectrum result:

1. **Instant baseline.** Every bin gets Rattlesnake's closed-form buzz solve — H^+ synthesis with cross-terms from a survey if available, zero if not. Effectively free.
2. **Targeted refinement.** The worst-predicted-error bins above `error_threshold_db` get the full SDP, replacing their buzz solution.
3. **Progressive.** Repeats every call, chipping away until the band is refined or good enough.
4. **FRF drift.** Already-refined bins whose H has moved past `frf_update_threshold` are re-solved with priority, capped at half the budget so they cannot starve step 2.
5. **Stale re-triage.** Leftover budget re-checks already-refined bins against the *current* H rather than H -drift magnitude — without this, once every bin is refined, step 2 never runs again and a degrading bin is orphaned on its stale solution forever.



`optimal_diagonal_control`. The budget is spent worst-error-first, with drifted bins taking priority.

The coherence cap reaches only SDP-refined bins.

The buzz baseline is left uncapped on purpose — it draws its cross-terms from an independent-drive survey, so it has no dependent-drive problem to begin with. With `max_bins_per_update = 20` across 901 lines, most bins carry the uncapped baseline. This is the most likely explanation for the measured null result: turning the cap off changed the overall error by 0.008 dB.

Parameters

POSITION	NAME	DEFAULT	MEANING
1	reg	1e-6	Tikhonov weight on $\ X\ _F$.
2	frf_update_threshold	0.05	Relative Frobenius change in H that triggers re-solving a refined bin.
3	max_bins_per_update	20	Hard cap on SDP solves per call.
4	error_threshold_db	1.0	Do not refine a bin already this close under buzz.
5	max_drive_coherence	0.95	Coherence cap, applied inside the SDP. 1.0 disables it.

3. optimal_diagonal_control_fast

subclass · optimal_diagonal_control_fast.py

Identical baseline and scheduler, inherited unchanged. The only override is how a single bin is solved: the SDP is replaced by an unconstrained Burer-Monteiro factored solve *whenever that is safe*.

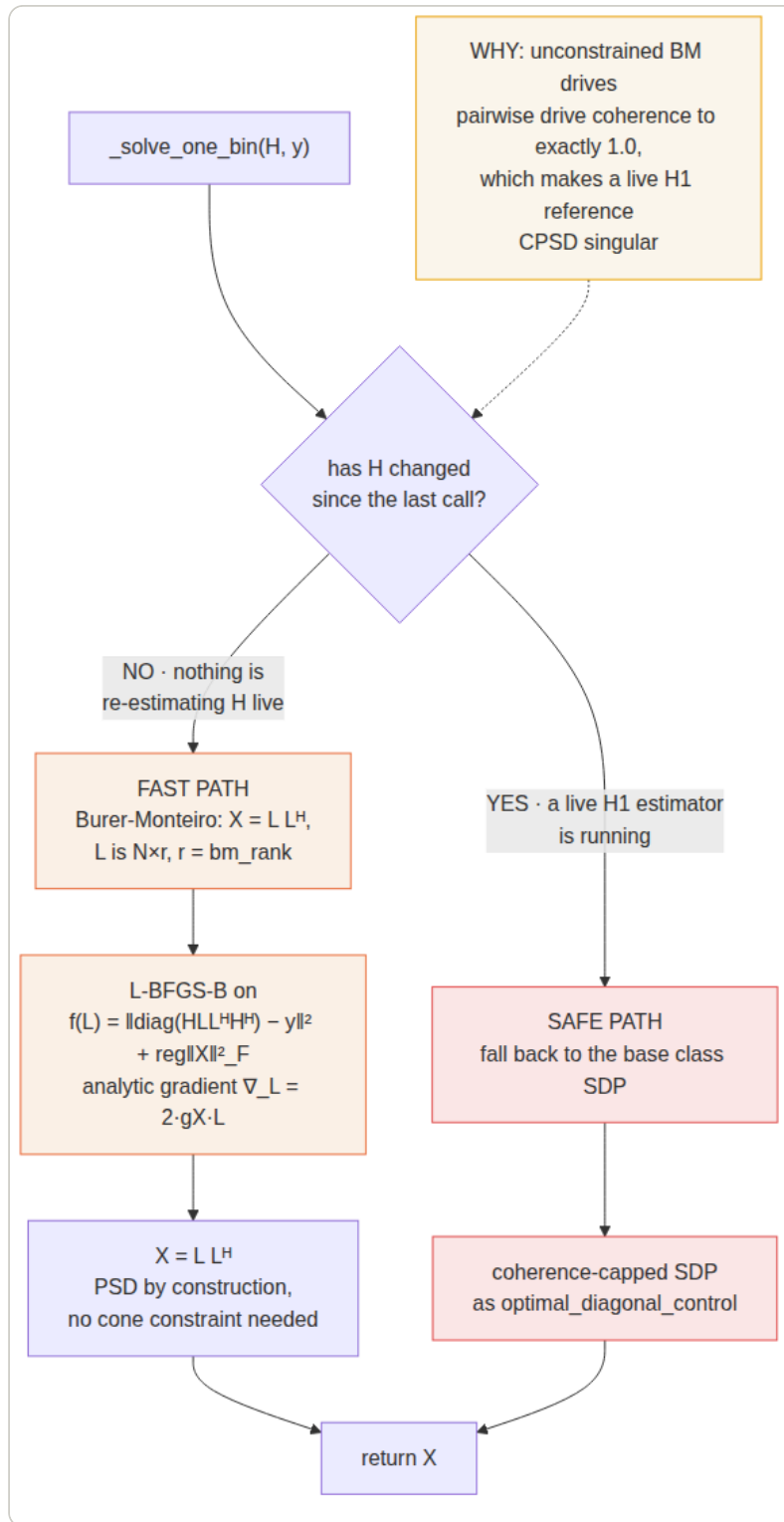
```
X = L LH, L ∈ ℂN×r, r = bm_rank

minimizeL || diag(H L LH HH) - y ||2 + reg||X||2F
∇X = 2 HH (resid ⊙ H) + 2·reg·X   ∇L = 2 ∇X L
L-BFGS-B, analytic gradient, maxiter 50
```

Factoring X as $L L^H$ makes positive semidefiniteness *structural* — the parametrisation is unconstrained, so no cone projection is needed and the solve is far cheaper. Benchmarked at a 12-drive/30-response scale: SDP ~105–119 ms/bin against ~5.4–5.7 ms/bin for the factored solve, roughly 18–20× faster, matching SDP accuracy to a mean of -0.02 dB.

The safety rule, and why it disabled the fast path here.

Left unconstrained, the factored solve drives pairwise drive coherence to *exactly 1.0* in essentially every bin — perfectly rank-deficient drives, which would make a live H1 estimator's reference CPSD matrix singular. Rather than trying to detect the "Update Transfer Function During Control" setting, which the call signature does not expose, the law watches the safety-relevant property directly: **has H changed since the last call?** If it has, every solve for the rest of that call uses the safe capped SDP instead. In this study FRF update was on, so H changed every call and the fast path never engaged — runs 05 and 06 are repeats of 03 and 04.



`optimal_diagonal_control_fast`. The branch at the top is the whole law; everything else is inherited.

Parameters

The same five as `optimal_diagonal_control`, plus:

POSITION	NAME	DEFAULT	MEANING
6	bm_rank	4	Factorisation rank r . With $M = 8$ the threshold below which spurious local minima generically appear is $\sqrt{(2M)} \approx 4$, so the default sits exactly on it; 6 was used in this study.

4. match_diagonal_congruence

class · diagonal_congruence_control.py

Generalises match_trace from one scalar to N independent knobs, while keeping the structural property that makes match_trace safe.

The generalisation

```
match_trace:  $X \leftarrow c \cdot X$  (one real scalar)
this law:  $X \leftarrow D X D$ ,  $D = \text{diag}(e^{u_1} \dots e^{u_N})$ ,  $u$  real
```

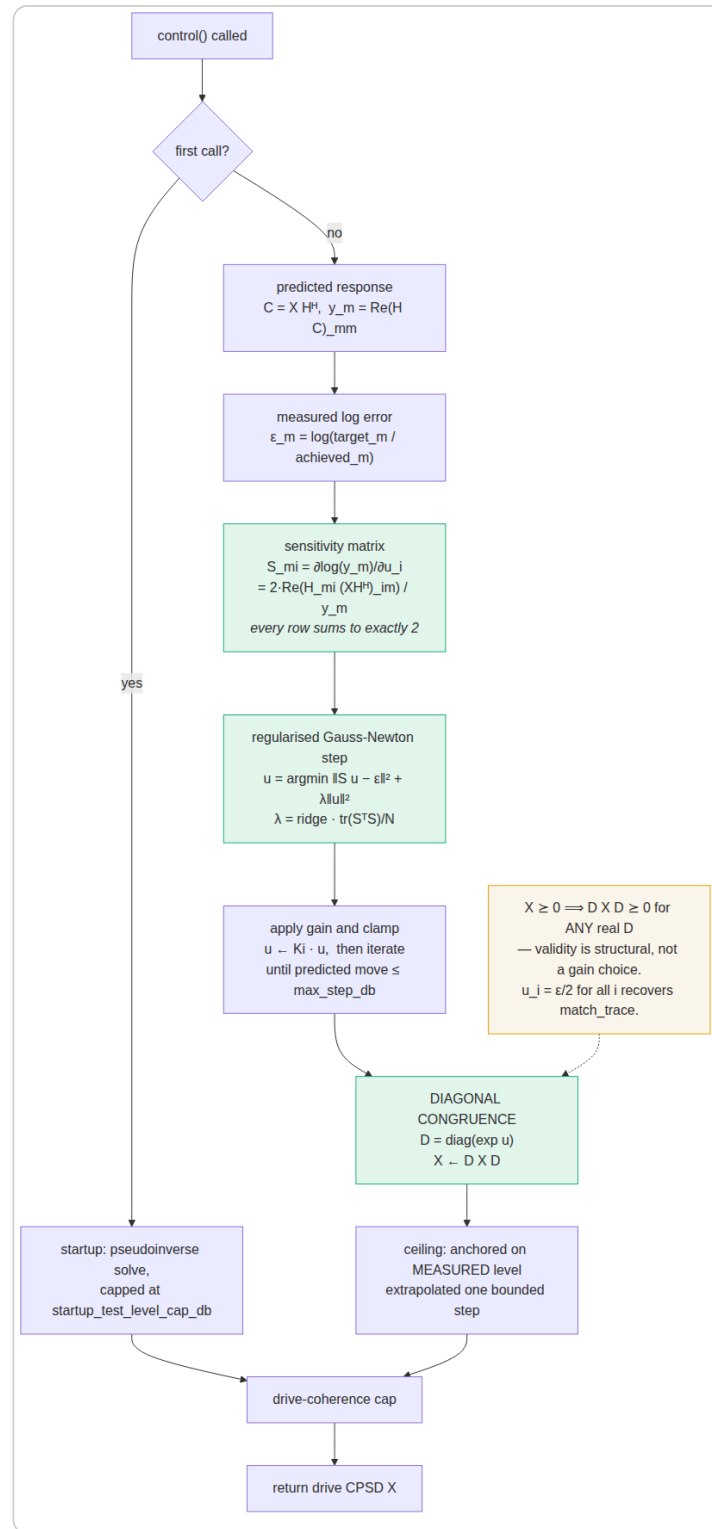
A diagonal congruence preserves positive semidefiniteness *structurally*: if $X \geq 0$ then $D X D \geq 0$ for **any** real diagonal D . So exactly as with match_trace, no choice of gains — however wrong the FRF, however noisy the measurement — can command a physically invalid drive. match_trace is the special case $D = \sqrt{c} \cdot I$.

Choosing the gains

```
 $y_m = (H X H^H)_{mm}$  predicted response
 $S_{mi} = \partial \log(y_m) / \partial u_i = 2 \operatorname{Re}(H_{mi} (X H^H)_{im}) / y_m$ 
 $\epsilon_m = \log(\text{target}_m / \text{achieved}_m)$  measured log error

 $u = \operatorname{argmin} \|S u - \epsilon\|^2 + \lambda \|u\|^2$ ,  $\lambda = \text{ridge} \cdot \operatorname{tr}(S^T S) / N$ 
one Gauss-Newton step on the log-domain problem, restricted to diagonal
congruence
```

Every row of S sums to exactly 2, because scaling all gains together by t scales X by e^{2t} and hence y by e^{2t} . The consequence is that when the error is the same on every channel, the solution is $u_i = \epsilon/2$ for all i — *the law reproduces match_trace's update exactly*. It degrades gracefully to the law it generalises.



`match_diagonal_congruence`. The congruence step is what guarantees validity; the Gauss-Newton step only chooses how far to move.

max_step_db bounds the response, not the gains.

An early version clamped u directly, which does not bound the outcome: rows of S sum to 2 but individual entries can be large and opposite-signed, so a 3 dB clamp on u produced a 38 dB swing in response. The clamp is now enforced on the exact predicted per-channel response move, iteratively, up to six passes.

Parameters

POSITION	NAME	DEFAULT	MEANING
1	rcond	1e-3	Pseudoinverse cutoff for the startup solve.
2	max_drive_coherence	0.95	Drive coherence cap; 1.0 disables.
3	startup_test_level_cap_db	-9.0	Startup ceiling, as match_trace.
4	Ki	0.5	Gain applied to the Gauss-Newton step.
5	max_step_db	3.0	Bound on the predicted per-channel response move per cycle.
6	ridge	0.1	Regularisation λ , relative to $\text{tr}(S^T S)/N$.
7	resonance_sensitivity	0.5	Half-power resonance detection sensitivity.
8	ceiling_db	12.0	Ceiling anchored on measured level, extrapolated one bounded step.
9	projected_target_file	none	Optional .npz whose target_diag replaces the raw specification as the target. Not used in this study — all four laws aimed at the same raw specification.

5. pseudoinverse_control

function · control_laws.py

The simplest law in the set and the oldest: one open-loop pseudoinverse solve against the raw specification, recomputed identically every cycle. No error feedback, no trace management, no startup guard. It is the baseline against which every other law's extra machinery earns its place.

Algorithm

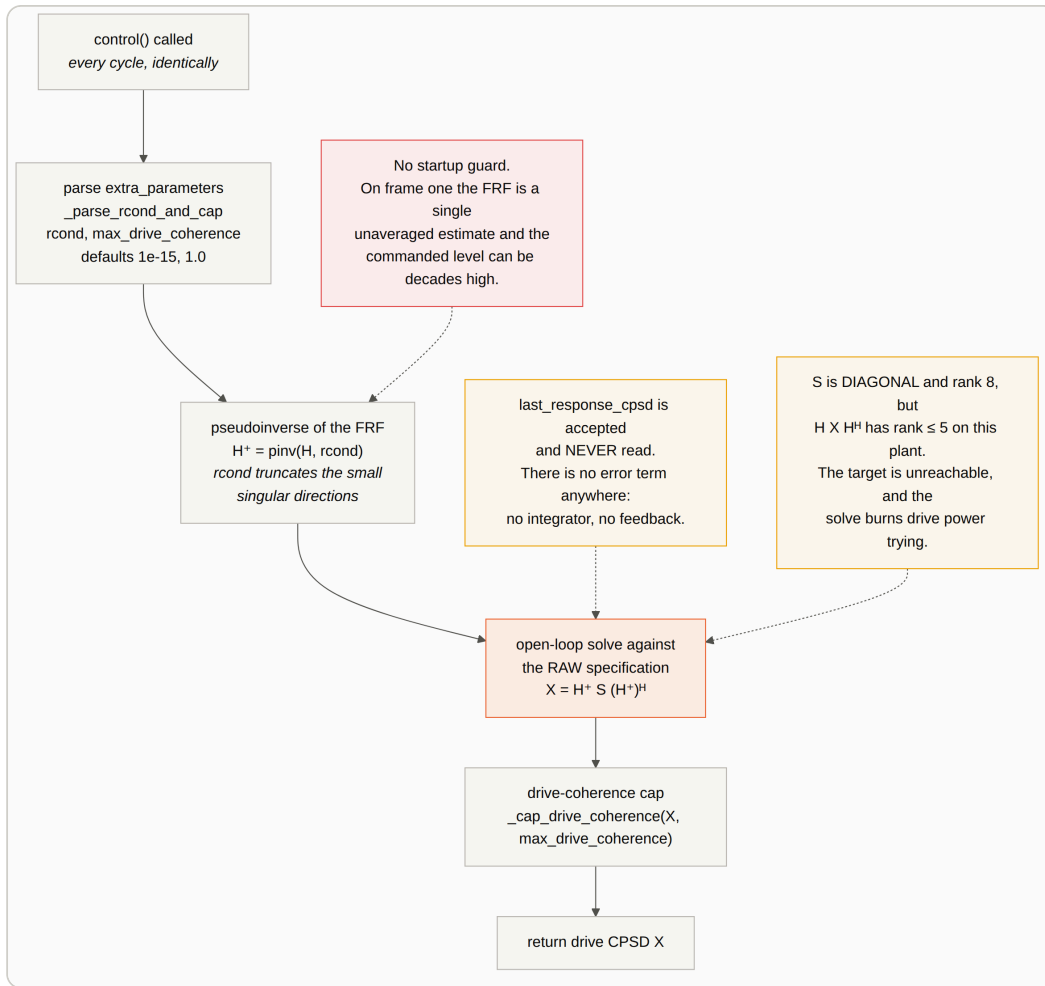
```
every call, identically
H+ = pinv(H, rcond)
X = H+ Gxx (H+)H
X ← cap(X, max_drive_coherence)
```

That is the entire law. `last_response_cpsd` appears in the signature and in the docstring — labelled “*Last Control Response for Error Correction*” — and is **never referenced in the body**. `last_output_cpsd` is not read either. There is no integrator and no error term of any kind, so the law cannot converge toward the specification; it lands wherever the current FRF estimate puts it and stays there.

The only thing that changes between cycles is `H`. With `update_tf_during_control = Y` the FRF is re-identified from live control data, so the law does adapt — but to a changing plant model, not to its own error. That distinction is the difference between an adaptive feedforward controller and a feedback one.

Why it is expensive

The flat specification is diagonal, hence rank 8, while $H X H^H$ has rank at most 5 on this plant. Inverting a maximally unreachable target with no trace or coherence management burns drive power trying to manufacture response components that do not exist in the column space. Measured: 5.38 V² uncapped, roughly seventy times the optimal-diagonal family, for a worse score.



pseudoinverse_control. One open-loop solve, recomputed identically every cycle.

Parameters

POSITION	NAME	DEFAULT	MEANING
1	rcond	1e-15	Pseudoinverse conditioning cutoff, applied on every call. 1e-3 was used in this study; on this plant that truncation is correct, since the sixth singular direction does not physically exist.
2	max_drive_coherence	1.0	Cap on pairwise drive coherence. 1.0 is a true no-op. Parsed by _parse_rcond_and_cap; there is no third parameter.

Open defect: no startup guard

This is the only law in the set without the first-cycle clamp added 2026-09-02. On frame one the FRF is a single unaveraged estimate under both averaging modes, and a poorly conditioned first-frame solve can command a level decades above anything the loop would settle at. match_trace, buzz and congruence all take startup_test_level_cap_db; this one does not.

Measured

RUN	CAP	RMS DB	% OUT	VS FLOOR	DRIVE V ²	LEVEL DB
09	0.95	10.47	42.1	9.55	5.449	+13.55
10	off	7.01	17.8	5.48	5.377	-1.28

Worst in the twelve-run matrix on both acceptance criteria with the cap on. The cap costs it **3.47 dB** — see section 7 for why the cap inverts its meaning for laws without feedback.

6. buzz_control

function · control_laws.py

pseudoinverse_control with one substitution, and that substitution is worth more than every feedback mechanism in the rest of the document. Before inverting, it replaces the specification's coherence and phase with the *measured* coherence and phase from the system-ID buzz test, keeping the specification's own auto-spectra.

Algorithm

```
the substitution ← match_coherence_phase(Gxx, Gbuzz)
Y2mn = |Gbuzz,mn|2 / (Gbuzz,mm Gbuzz,nn)  measured coherence
φmn = ∠Gbuzz,mn  measured phase
am = Gxx,mm  specified auto-spectra
S'mn = eiφmn √(Y2mn am an)

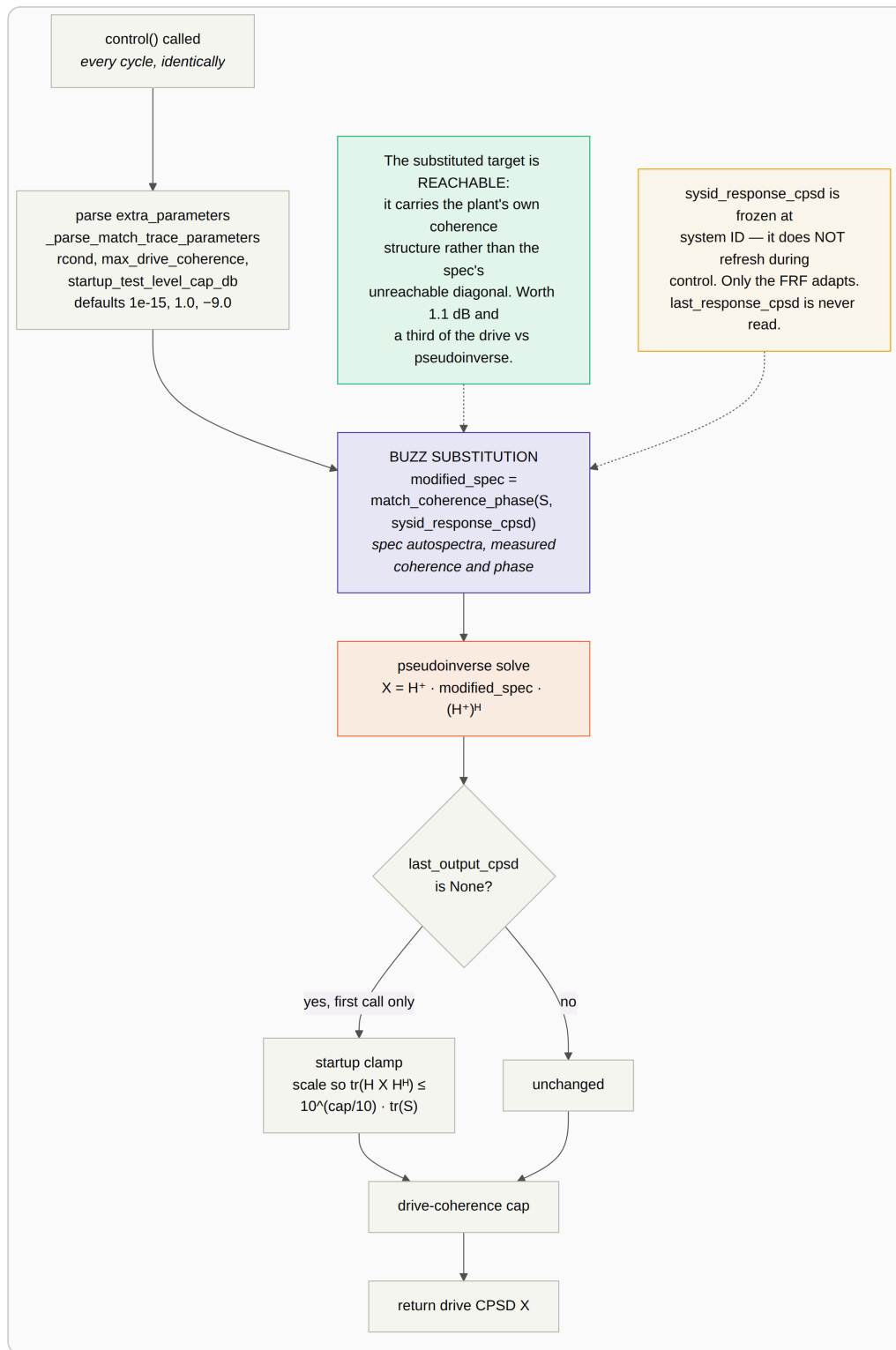
then the same open-loop solve
X = H+ S' (H+)H
first call only
X ← min(1, 10cap/10 tr(Gxx) / tr(H X HH)) · X
X ← cap(X, max_drive_coherence)
```

Why the substitution matters. The specification's implied coherence is zero everywhere — eight mutually uncorrelated responses — which a rank-5 plant cannot produce at all. The measured coherence, by construction, *is* something the plant produces. So buzz inverts a target that lies close to the achievable set while pseudoinverse inverts one that lies far outside it. Same solver, same conditioning, same cap: 1.1 dB better and a third of the drive.

Like pseudoinverse_control, it has **no error feedback**: last_response_cpsd is accepted and never read, and last_output_cpsd is used only as a sentinel to fire the startup clamp once. It also has a second, less obvious limitation — the two inputs refresh on different schedules.

The buzz target is frozen; only the FRF adapts

In random_vibration_sys_id_data_analysis.py, update_tf_during_control swaps in the re-identified control_frf. But sysid_response_cpsd — the source of the coherence and phase being matched — is assigned only in run_sysid_transfer_function and is never refreshed during control. The target shape is fixed for the whole run.



buzz_control. The substitution happens before the solve; everything after it is pseudoinverse_control.

Parameters

POSITION	NAME	DEFAULT	MEANING
1	rcond	1e-15	Pseudoinverse conditioning cutoff, applied on every call. 1e-3 was used in this study.
2	max_drive_coherence	1.0	Cap on pairwise drive coherence. 1.0 is a true no-op.
3	startup_test_level_cap_db	-9.0	Ceiling on the first command only. Unlike match_trace there is no separate steady-state branch to guard, so every later call is completely unaffected.

Shares `_parse_match_trace_parameters` with `match_trace_pseudoinverse`, so the parameter string is identical in form.

Measured

RUN	CAP	RMS DB	% OUT	VS FLOOR	DRIVE V^2	LEVEL DB
11	0.95	7.90	30.2	6.75	0.981	+9.38
12	off	5.91	13.6	4.20	1.473	-0.30

Uncapped, `buzz_control` finishes **0.68 dB behind the best run in the entire twelve-run matrix** and ahead of both `match_trace` runs — with no error feedback whatsoever. On this plant the 5.2–5.9 dB cluster is set by reachability and identification quality, not by loop design; what separates the laws is how sensibly each picks a target inside the reachable set.

The generator and class forms are not equivalent

`control_laws.py` also contains `buzz_control_generator` and `buzz_control_class`. Neither has the drive-coherence cap, neither takes `rcond`, and neither has the startup guard — they call a bare `np.linalg.pinv`. The generator caches `modified_spec` on its first call and never recomputes it. `buzz_control_class.control()` accepts `last_response_cpsd` and never references it, replaying one open-loop solve forever; its specification updates only through `system_id_update`. Both were excluded from the comparison deliberately.

7. The coherence cap means opposite things to the two families

`_cap_drive_coherence` shrinks any off-diagonal drive term whose pairwise coherence exceeds the cap, preserves every diagonal value, then re-projects onto the PSD cone by clipping negative eigenvalues. Applied to a *feedback* law it acts as a constraint the loop re-converges around. Applied to an *open-loop* law it is pure damage, because there is nothing to re-converge with.

LAW	LOOP	RMS CAP ON	RMS CAP OFF	Δ	LEVEL ON	LEVEL OFF
match_diagonal_congruence	feedback	5.31	5.23	-0.08	+2.85	+2.78
optimal_diagonal_control	feedback	5.62	5.62	-0.01	+0.08	+0.41
optimal_diagonal_control_fast	feedback	5.64	5.66	+0.02	+0.26	+0.06
match_trace_pseudoinverse	feedback	8.22	7.21	-1.01	+0.09	+0.05
buzz_control	open	7.90	5.91	-1.99	+9.38	-0.30
pseudoinverse_control	open	10.47	7.01	-3.47	+13.55	-1.28

The mechanism. Both open-loop solves are strongly rank-concentrated — eigenvalue participation ratio 1.0–1.1 out of 6 — so the six shakers act as essentially one source and the responses cancel by precise inter-drive phasing. That phasing lives entirely in the off-diagonal terms. The cap shrinks them while preserving the diagonal, so drive power is barely changed but the cancellation is destroyed and responses add incoherently: the level jumps roughly 20 dB. The residual is *shape* error, not level — the best possible single rescale of run 09's entire drive recovers only 0.30 dB.

Operational consequence

For `pseudoinverse_control` and `buzz_control`, **cap off (1.0) is the correct setting**, not the reckless one. This is the reverse of the feedback-law intuition and nothing in the parameter name or the docstrings warns of it. Confirmed in reverse: the best achievable drive found by the floor solver sits above 0.95 coherence on 49 % of its pairs, so the cap constrains the solution away from the optimum by construction.

8. achievable_response

not a control law · achievable_response.py

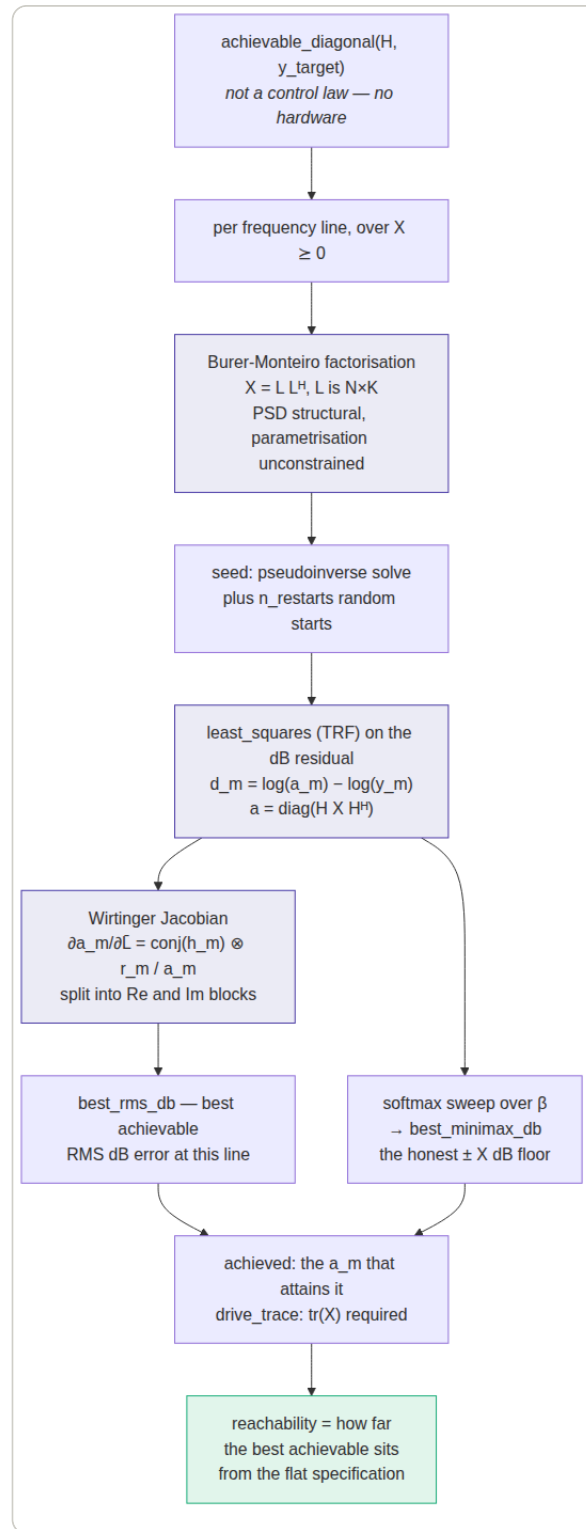
The pseudo control law. It drives no hardware. Given an FRF it answers the question every comparison needs a denominator for: *how much of my response error is the control law's fault, and how much is simply not reachable with this structure and these shakers?*

per frequency line, over $X \geq 0$:

best_rms_db $\min \sqrt{(\text{mean}_m (10 \log_{10}(a_m/y_m))^2)}$
best_minimax_db $\min \max_m |10 \log_{10}(a_m/y_m)|$ the honest $\pm X$ dB floor
achieved the $a_m = \text{diag}(H X H^H)$ that attains it
drive_trace $\text{tr}(X)$ required – feasibility against real shaker limits

Solved by the same Burer-Monteiro factorisation the fast law uses, $X = L L^H$, but minimising the residual in the **dB domain** rather than the linear one — dB error is the quantity a vibration test is judged by, and a linear residual over-weights loud channels. The Jacobian is built with Wirtinger calculus:

$d_m = \log(a_m) - \log(y_m)$ residual
 $\partial a_m / \partial \bar{L} = \text{conj}(h_m) \otimes r_m / a_m$
 $J = [\text{Re}(G) \mid \text{Im}(G)]$ real and imaginary blocks, $2NK$ columns
`least_squares(method='trf')` – not 'lm', which refuses $m < n$



achievable_response. The output is a bound, not a command — but it is what makes "how good is this law" a well-posed question.

Parameters

NAME	DEFAULT	MEANING
rank	N	Factorisation rank K. N = 6 here, above the $\sqrt{(2M)} \approx 4$ threshold.
n_restarts	2	Random restarts beyond the pseudoinverse seed. restart_spread_db reports landscape ruggedness, not unreliability.
rcond	1e-8	Conditioning cutoff for the <i>starting point</i> only; does not constrain the answer.
restrict_rcond	none	Truncates the drive subspace — the honest way to ask "what is achievable with well-conditioned drives?" 1e-3 costs 0.74 dB while saving four and a half orders of magnitude of drive power.
compute_minimax	True	Also solve the minimax objective via a softmax sweep over β .
max_nfev	400	Function-evaluation budget per line.

9. Lineage and measured results

Each law solves the same problem with a different guarantee of validity, and that choice is what determines both its power and its limits:

LAW	HOW $X \geq 0$ IS GUARANTEED	DEGREES OF FREEDOM PER LINE	RMS DB	% OUT	DRIVE TRACE
match_trace_pseudoinverse	uniform positive scalar	1	8.22	27.5	0.12
optimal_diagonal_control	explicit SDP cone constraint	N^2 (full X)	5.62	11.2	0.075
optimal_diagonal_control_fast	structural, $X = L L^H$	$2Nr$	5.64	10.9	0.076
match_diagonal_congruence	structural, $D \times D$	N	5.31	19.8	0.047
pseudoinverse_control	none needed — X is a congruence of $G_{xx} \geq 0$	0 (open loop)	10.47	42.1	5.449
buzz_control	none needed — X is a congruence of $S' \geq 0$	0 (open loop)	7.90	30.2	0.981

Results are the **cap-on** runs from the 15 September twelve-run comparison, linear plant, against the flat specification. Cap-on is the wrong setting for the two open-loop laws (section 7), so their rows understate them badly: uncapped they measure 7.01 and 5.91 dB with 17.8 % and 13.6 % of lines out. Scored instead against what is *achievable*, congruence leads at 3.48 dB, the optimal-diagonal family sits at a flat 3.99, buzz at 4.20, pseudoinverse at 5.48 and match_trace at 5.78.

The two open-loop laws need no positive-semidefiniteness machinery at all: $X = H^+ S (H^+)^H$ is a congruence transform of a PSD matrix, so validity is automatic. Every guarantee in the table above buys the ability to *shape* the drive, not the ability to keep it valid — and on this plant that shaping is worth between 0 and 1.1 dB against a well-chosen target, which is the campaign's least comfortable result.

Open defects.

Neither optimal-diagonal law has a startup level cap — match_trace and congruence both take startup_test_level_cap_db, but the optimal-diagonal laws solve against the full specification and command it immediately. The fast law's central claim, that Burer-Monteiro at rank 6 matches the SDP, remains untested on this system because FRF update disabled its fast path. pseudoinverse_control has no startup guard either, and neither buzz_control_generator nor buzz_control_class has the coherence cap or the startup guard.

Algorithms read from the implementations in control_laws/ at commit e6f961d6. Flow charts are Mermaid; sources are committed alongside this document so they stay editable as text.